

Foundations of Neural architecture search

[Neural architecture search]

1.0 Content Block

Evolution An alternative approach to NAS is based on evolutionary algorithms, which has been employed by several groups. An Evolutionary Algorithm for Neural Architecture Search generally performs the following procedure. First a pool consisting of different candidate architectures along with their validation scores (fitness) is initialised. At each step the architectures in the candidate pool are mutated (e.g.: 3x3 convolution instead of a 5x5 convolution). Next the new architectures are trained from scratch for a few epochs and their validation scores are obtained. This is followed by replacing the lowest scoring architectures in the candidate pool with the better, newer architectures. This procedure is repeated multiple times and thus the candidate pool is refined over time. Mutations in the context of evolving ANNs are operations such as adding or removing a layer, which include changing the type of a layer (e.g., from convolution to pooling), changing the hyperparameters of a layer, or changing the training hyperparameters. On CIFAR-10 and ImageNet, evolution and RL performed comparably, while both slightly outperformed random search.

2.0 Content Block

Hill-climbing Another group used a hill climbing procedure that applies network morphisms, followed by short cosine-annealing optimization runs. The approach yielded competitive results, requiring resources on the same order of magnitude as training a single network. E.g., on CIFAR-10, the method designed and trained a network with an error rate below 5% in 12 hours on a single GPU.

3.0 Content Block

Wistuba, Martin; Rawat, Ambrish; Pedapati, Tejaswini (2019-05-04). "A Survey on Neural Architecture Search". arXiv:1905.01392 [cs.LG]. Elsken, Thomas; Metzen, Jan Hendrik; Hutter, Frank (August 8, 2019). "Neural Architecture Search: A Survey". Journal of Machine Learning Research. 20 (55): 1–21. arXiv:1808.05377. Liu, Yuqiao; Sun, Yanan; Xue, Bing; Zhang, Mengjie; Yen, Gary G; Tan, Kay Chen (2021). "A Survey on Evolutionary Neural Architecture Search". IEEE Transactions on Neural Networks and Learning Systems. 34 (2): 1–21. arXiv:2008.10937. doi:10.1109/TNNLS.2021.3100554. PMID 34357870. S2CID 221293236. White, Colin; Safari, Mahmoud; Sukthanker, Rhea; Ru, Binxin; Elsken, Thomas; Zela, Arber; Dey, Debadeepta; Hutter, Frank (2023-01-25). "Neural Architecture Search: Insights from 1000 Papers". arXiv:2301.08727 [cs.LG].

4.0 Content Block

Multi-objective search While most approaches solely focus on finding architecture with maximal predictive performance, for most practical applications other objectives are relevant, such as memory consumption, model size or inference time (i.e., the time required to obtain a prediction). Because of that, researchers created a multi-objective search. LEMONADE is an evolutionary algorithm that adopted Lamarckism to efficiently optimize multiple objectives. In every generation, child networks are generated to improve the Pareto frontier with respect to the current population of ANNs. Neural Architect is claimed to be a resource-aware multi-objective RL-based NAS with network embedding and performance prediction. Network embedding encodes an existing network to a trainable embedding vector. Based on the embedding, a controller network generates transformations of the target network. A multi-objective reward function considers network accuracy, computational resource and training time. The reward is predicted by multiple performance simulation networks that are pre-trained or co-trained with the controller network. The controller network is trained via policy gradient. Following a modification, the resulting candidate

network is evaluated by both an accuracy network and a training time network. The results are combined by a reward engine that passes its output back to the controller network.

5.0 Content Block

Reinforcement learning Reinforcement learning (RL) can underpin a NAS search strategy. Barret Zoph and Quoc Viet Le applied NAS with RL targeting the CIFAR-10 dataset and achieved a network architecture that rivals the best manually-designed architecture for accuracy, with an error rate of 3.65, 0.09 percent better and 1.05x faster than a related hand-designed model. On the Penn Treebank dataset, that model composed a recurrent cell that outperforms LSTM, reaching a test set perplexity of 62.4, or 3.6 perplexity better than the prior leading system. On the PTB character language modeling task it achieved bits per character of 1.214. Learning a model architecture directly on a large dataset can be a lengthy process. NASNet addressed this issue by transferring a building block designed for a small dataset to a larger dataset. The design was constrained to use two types of convolutional cells to return feature maps that serve two main functions when convoluting an input feature map: normal cells that return maps of the same extent (height and width) and reduction cells in which the returned feature map height and width is reduced by a factor of two. For the reduction cell, the initial operation applied to the cell's inputs uses a stride of two (to reduce the height and width). The learned aspect of the design included elements such as which lower layer(s) each higher layer took as input, the transformations applied at that layer and to merge multiple outputs at each layer. In the studied example, the best convolutional layer (or "cell") was designed for the CIFAR-10 dataset and then applied to the ImageNet dataset by stacking copies of this cell, each with its own parameters. The approach yielded accuracy of 82.7% top-1 and 96.2% top-5. This exceeded the best human-invented architectures at a cost of 9 billion fewer FLOPS—a reduction of 28%. The system continued to exceed the manually-designed alternative at varying computation levels. The image features learned from image classification can be transferred to other computer vision problems. E.g., for object detection, the learned cells integrated with the Faster-RCNN framework improved performance by 4.0% on the COCO dataset. In the so-called Efficient Neural Architecture Search (ENAS), a controller discovers architectures by learning to search for an optimal subgraph within a large graph. The controller is trained with policy gradient to select a subgraph that maximizes the validation set's expected reward. The model corresponding to the subgraph is trained to minimize a canonical cross entropy loss. Multiple child models share parameters, ENAS requires fewer GPU-hours than other approaches and 1000-fold less than "standard" NAS. On CIFAR-10, the ENAS design achieved a test error of 2.89%, comparable to NASNet. On Penn Treebank, the ENAS design reached test perplexity of 55.8.

6.0 Content Block

Bayesian optimization Bayesian Optimization (BO), which has proven to be an efficient method for hyperparameter optimization, can also be applied to NAS. In this context, the objective function maps an architecture to its validation error after being trained for a number of epochs. At each iteration, BO uses a surrogate to model this objective function based on previously obtained architectures and their validation errors. One then chooses the next architecture to evaluate by maximizing an acquisition function, such as expected improvement, which provides a balance between exploration and exploitation. Acquisition function maximization and objective function evaluation are often computationally expensive for NAS, and make the application of BO challenging in this context. Recently, BANANAS has achieved promising results in this direction by introducing a high-performing instantiation of BO coupled to a neural predictor.